

Accelerating RL Post-Training Rollouts via System-Integrated Speculative Decoding

Hayate Iso, Tiyasa Mitra, Sudipta Mondal, Rasoul Shafipour, Venmugil Elango, Terry Kong, Yuki Huang, Seonjin Na, Izzy Putterman, Benjamin Chislett, Maor Ashkenazi, Joseph Guman, Gerald Shen, Tugrul Konuk, Ashwath Aithal, Ritika Borkar, Ran Zilberstein, Bitu Rouhani

Abstract. RL post-training of frontier language models is increasingly bottlenecked by autoregressive rollout generation, making rollout acceleration a central systems challenge. Many existing efficiency methods improve throughput by changing the rollout or optimization regime, for example, through off-policy execution, replay, or lower-precision generation. We study speculative decoding as a lossless acceleration primitive for RL rollouts that preserves the target model’s output distribution. We implement speculative decoding in NeMo-RL with a vLLM backend, supporting both synchronous and asynchronous pipelines and enabling speculation during RL rollouts. This benefit is realizable across speculation mechanisms, such as pretrained MTP heads, small external draft models or even techniques such as Eagle3, which are traditionally applied after RL phase. This yields a deployment path for state-of-the-art speculative decoding inside RL training. In a reasoning post-training workload at 8B scale under synchronous RL, speculative decoding improves rollout throughput by $1.8\times$. Using a high-fidelity performance simulator, we project that combining speculative decoding with asynchronous RL yields up to $2.5\times$ end-to-end training speedup at 235B scale.

1. Introduction

Reinforcement learning (RL) post-training of frontier language models is increasingly bottlenecked by autoregressive rollout generation. RL is widely used to improve reasoning performance in language models, especially on mathematical and other verifiable tasks (Shao et al., 2024; DeepSeek-AI, 2025a), and in these reasoning-oriented workloads overall training time is dominated by rollout generation rather than by gradient computation: across recent large-scale RL systems, rollout generation is routinely the single largest wall-clock component (Shen et al., 2024; Hu et al., 2025; Noukhovitch et al., 2025; Piché et al., 2025; Meta GenAI, 2025). The same issue is emerging in agentic RL, where long-horizon tasks require many repeated model calls across tool-use, retrieval, and web-interaction steps and further amplify the cost of every decoded token (Jin et al., 2025; Qian et al., 2025; Wang et al., 2025b; Qi et al., 2025). This makes generation efficiency increasingly important, and speculative decoding offers a way to improve it without changing the verifier-side training semantics.

Several approaches reduce this cost by changing the training dynamics. Asynchronous execution overlaps generation with learning but introduces policy lag (Noukhovitch et al., 2025; Piché et al., 2025; Meta GenAI, 2025). Off-policy replay and importance-sampling corrections reuse stale trajectories (Li et al., 2025a; Zheng et al., 2025a; Wang et al., 2025a; Sheng et al., 2026). Lower-precision rollouts reduce compute but introduce distribution mismatch (Xi et al., 2026; Qiu et al., 2026; Li et al., 2026). Selective prompt filtering skips uninformative prompts (Zheng et al., 2025b). These methods are effective, but each changes the sampling or optimization semantics of the original problem.

Speculative decoding offers a different trade-off. A draft model proposes several tokens at once, and the target model verifies them through a rejection procedure that preserves its output distribution

(Leviathan et al., 2023; Chen et al., 2023; Cai et al., 2024; Li et al., 2024; Gloeckle et al., 2024). Applied to RL rollouts, this makes generation faster without changing what distribution the trajectories are drawn from, a property that matters because the RL training signal depends on the policy’s own samples. In practice, however, the gains depend on how well the draft matches the policy, how many tokens are proposed per step, and what fraction of the RL step is spent on generation.

We study speculative decoding as a rollout acceleration primitive integrated into an open-source RL training framework. We implement it in NeMo RL with a vLLM serving backend, supporting a general drafting path based on EAGLE-3 (Li et al., 2024, 2025b) that works with any pretrained model, as well as a native path for models with built-in multi-token prediction heads (Gloeckle et al., 2024). The integration handles weight synchronization between the learner and the rollout engine, optional online draft adaptation, and composition with both synchronous and asynchronous RL pipelines.

Our contributions are:

- We integrate speculative decoding into NeMo RL as a rollout primitive that preserves verifier-exact training semantics, and describe the system requirements (weight synchronization, draft coherence, and stage-level telemetry) needed for a working deployment.
- We evaluate the integration on 8B-scale reasoning workloads under synchronous RL and study the operational choices (draft initialization, draft length, online adaptation, and interaction with asynchronous execution) that determine realized speedup.
- We characterize the design space of speculative decoding for RL post-training, quantifying how end-to-end speedup varies with model scale, GPU count, generation share, and policy lag, and identify the operating regimes in which the technique yields meaningful gains at deployment scale.

2. Speculative Decoding for RL Rollouts

2.1. Learning speed and lossless acceleration

We view RL progress through a simple decomposition (Piché et al., 2025):

$$\text{learning speed} = \text{effectiveness} \times \text{throughput}.$$

Here, *throughput* is how much rollout and training work the system completes per unit time, and *effectiveness* is how much useful learning signal is extracted from that work. In RL post-training, the training distribution is generated online by the policy itself, so a mechanism that increases throughput by perturbing the sampling distribution may reduce effectiveness. Methods such as asynchronous execution, off-policy replay, and low-precision rollouts each trade effectiveness for throughput in different ways. Speculative decoding targets throughput without changing the sampling distribution: the rejection procedure guarantees that rollouts follow the verifier policy, so effectiveness is preserved by construction.

2.2. Speedup analysis

In NeMo RL, a synchronous RL step decomposes as

$$T_{\text{step}} = T_{\text{data}} + T_{\text{prepare}} + T_{\text{gen}} + T_{\text{logprob}} + T_{\text{train}},$$

where T_{prepare} covers weight synchronization and rollout backend preparation, T_{gen} covers rollout generation, T_{logprob} is log-probability recomputation under the current policy, and T_{train} includes

advantage computation and policy optimization. Speculative decoding targets only T_{gen} , and within generation, only the autoregressive decode phase (not prefill). The technique is therefore most useful when generation dominates the RL step and generation is decode-heavy.

The step-level speedup is bounded by

$$S_{\text{step}} \leq \frac{1}{R_{\text{gen}}/\alpha + (1 - R_{\text{gen}})},$$

where $R_{\text{gen}} = T_{\text{gen}}/T_{\text{step}}$ is the generation share and α is the mean acceptance length, defined as the average number of tokens produced per speculation step. This bound follows from Amdahl’s law under the assumption that each speculation step costs the same as one autoregressive forward pass. In practice, draft model overhead, prefill time, and batching effects reduce the realized speedup below this bound. The bound nevertheless captures the central trade-off: speculation helps only to the extent that (i) generation dominates step time and (ii) the acceptance length is high enough to outweigh verification cost.

2.3. System integration

Integrating speculative decoding into an RL training loop goes beyond adding a draft model to the serving backend. As the policy updates each step, the rollout engine must receive new weights, and the draft model must remain aligned with the current policy. Log-probabilities, KL penalties, and the policy loss must be computed against the verifier (target) policy, not the draft; otherwise, speculative decoding would alter the optimization target. These requirements distinguish RL integration from standalone inference serving, where model weights are fixed.

Figure 1 shows the architecture. The vLLM backend generates rollout trajectories using speculative decoding. The policy model (MegatronLM) then runs the forward pass used to compute the GRPO policy loss (Shao et al., 2024). When online draft adaptation is enabled, that same forward pass also produces cached hidden states and log-probabilities, which are reused for EAGLE-3 draft supervision rather than recomputed in a separate policy pass. The hidden-state cache is routed through a gradient-detached pathway to the draft head, so that draft training does not interfere with the policy gradient signal.

The system supports two drafting paths. The *general path*, based on EAGLE-3 (Li et al., 2024, 2025b), works with any pretrained model without requiring native multi-token-prediction support. A *native path* is also available for models that ship with built-in MTP heads (Gloeckle et al., 2024), where the model’s own auxiliary heads serve as the draft mechanism.

Speculative decoding composes with both synchronous and asynchronous RL. In synchronous mode, it directly reduces rollout latency. In asynchronous mode, generation overlaps with other pipeline stages, reducing the effective generation share on the critical path and correspondingly reducing the speedup from speculation. The two mechanisms are complementary: speculation makes each rollout cheaper, while async overlap hides remaining generation cost behind training and log-probability computation.

3. Experiments

3.1. Setup

Policy. We evaluate GRPO-based RL post-training (Shao et al., 2024) on mathematical reasoning in two settings. *RL-Think* continues training a reasoning-capable model to refine thinking traces,

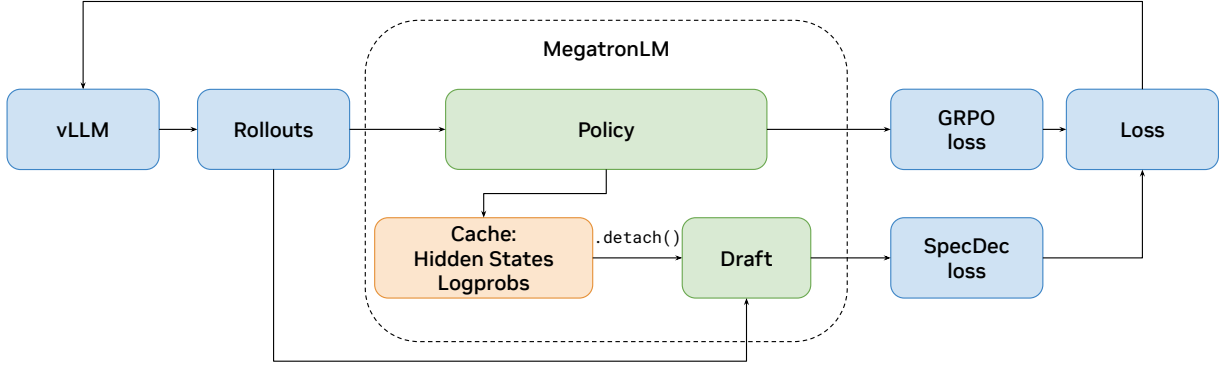


Figure 1 | System overview of NeMo RL with speculative decoding. The vLLM backend produces rollout trajectories, and the policy model (MegatronLM) runs the verifier forward pass for the GRPO policy loss. When online adaptation is enabled, hidden states and log-probabilities from that same pass are cached and reused by a separate speculative-decoding loss for the draft head through a gradient-detached pathway (`.detach()`). The detached boundary ensures that draft training does not alter the policy gradient signal.

while *RL-Zero* starts directly from the corresponding base model. RL-Think uses Qwen3-8B¹, and RL-Zero uses Qwen3-8B-Base² (Qwen Team, 2025). For training we use DAPO-Math-17K³ (Yu et al., 2025), and for validation we report accuracy on AIME-2024⁴.

Draft. Our approach leverages the general EAGLE-3 drafting path (Li et al., 2024, 2025b), which enables speculative decoding for models that do not have native MTP heads (Gloeckle et al., 2024; DeepSeek-AI, 2025a) or specially distilled external draft models. We deliberately focus the empirical study on EAGLE-3 as the harder case: it requires training and maintaining an external drafter, and must be kept aligned with a moving policy throughout RL. The system-level findings below (draft initialization, draft length, online adaptation, and composition with asynchronous execution) carry over directly to the native MTP path (Gloeckle et al., 2024), where the model’s built-in auxiliary heads play the role of the draft and the same alignment and configuration trade-offs apply. In the main experiments, we initialize the draft offline using responses generated by the policy on the training prompts, aligning the draft to the actual rollout distribution. Unless otherwise specified, we adopt a draft length of $k = 3$ and maintain the EAGLE-3 draft weights fixed during RL. Section 3.3 further examines the impact of draft initialization, draft length, and online draft adaptation.

Configurations. Experiments run on 8 GB200 NVL72 nodes with 4 GB200 GPUs per node, each with 186 GB of HBM3E memory, connected by fifth-generation NVLink. We also compare autoregressive decoding against EAGLE-3 and n -gram drafting under otherwise matched RL configurations.

3.2. Main results

We begin by examining where time goes in a typical RL step, since this determines the ceiling for any rollout-side optimization. Table 1 shows the per-stage breakdown. Generation is the single largest stage, roughly 65–72% of step time in both settings, and is the only stage that speculative decoding directly accelerates. Log-probability recomputation and training together account for another ~27–33% and are unchanged regardless of how tokens are generated.

¹<https://huggingface.co/Qwen/Qwen3-8B>

²<https://huggingface.co/Qwen/Qwen3-8B-Base>

³<https://huggingface.co/datasets/BytedTsinghua-SIA/DAPO-Math-17k>

⁴<https://huggingface.co/datasets/BytedTsinghua-SIA/AIME-2024>

Stage	RL-Think		RL-Zero	
	AR (s)	Spec (s)	AR (s)	Spec (s)
Data	0.3	0.2	0.2	0.2
Prepare	2.1	1.6	1.9	2.1
Generation	133.6	87.0	100.0	56.6
Log-prob	17.9	18.1	17.8	18.1
Training	31.4	30.5	31.3	30.5
Overall	185.3	137.4 (1.35 $\times$)	151.2	107.5 (1.41 $\times$)

Table 1 | Mean step-time breakdown per RL step. Generation accounts for 65–72% of step time in the autoregressive baseline; speculative decoding reduces it by 1.77 $\times$ on RL-Zero and 1.54 $\times$ on RL-Think. Log-prob recomputation and training are unchanged, capping total step speedup at 1.41 $\times$ and 1.35 $\times$ respectively.

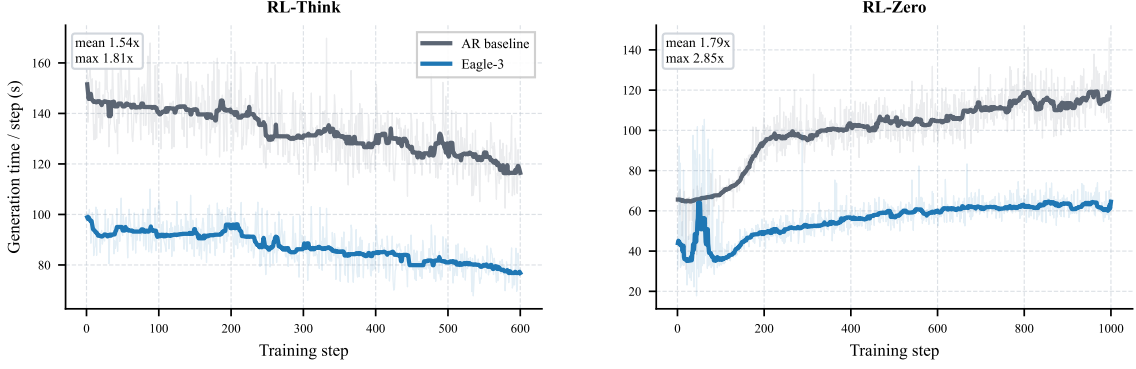
Given that generation dominates, Table 2 shows how speculative decoding targets this bottleneck. EAGLE-3 reduces generation latency from 100.0s to 56.6s on RL-Zero (1.8 $\times$) and from 133.6s to 87.0s on RL-Think (1.5 $\times$). These generation-side gains translate to the 1.41 $\times$ and 1.35 $\times$ overall step speedups in Table 1, bounded by the non-generation stages that speculative decoding cannot touch.

We also compare against n -gram drafting, a model-free speculative baseline. Despite achieving non-trivial acceptance lengths (2.47 on RL-Zero, 2.05 on RL-Think), n -gram drafting is *slower* than the autoregressive baseline in both settings. This confirms that positive acceptance alone is insufficient; verification overhead can erase the benefit entirely.

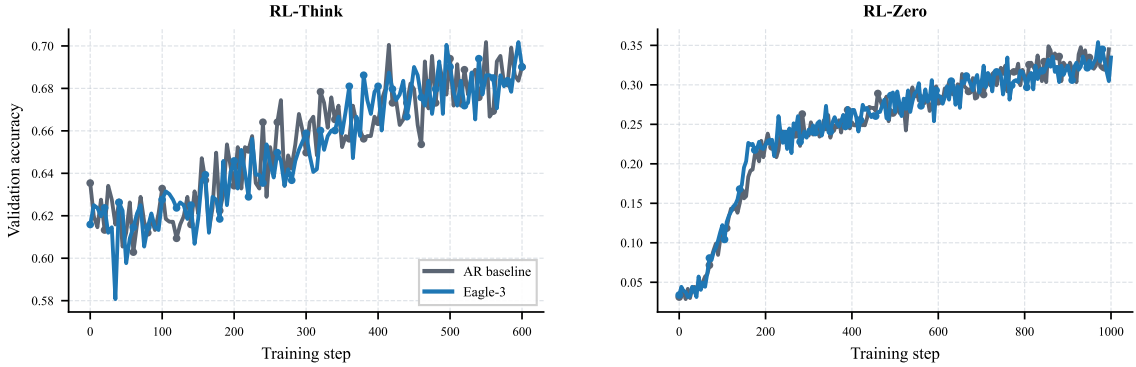
Method	Accept. len.	Gen latency / step (s)	Speedup
RL-Zero			
Autoregressive	–	100.0	1.0 $\times$
n -gram	2.47	140.2	0.7 $\times$
EAGLE-3	3.32	56.6	1.8 $\times$
RL-Think			
Autoregressive	–	133.6	1.0 $\times$
n -gram	2.05	262.9	0.5 $\times$
EAGLE-3	2.77	87.0	1.5 $\times$

Table 2 | Rollout generation comparison. EAGLE-3 substantially reduces generation latency, while n -gram drafting is slower than autoregressive decoding despite non-trivial acceptance lengths.

Figure 2 confirms these findings over the full training run. Generation latency under EAGLE-3 remains consistently below the autoregressive baseline, with mean speedups of 1.54 $\times$ on RL-Think and 1.79 $\times$ on RL-Zero (Figure 2a). Notably, the RL-Zero baseline latency rises sharply during the first ~ 100 steps as the policy transitions from short outputs to longer reasoning traces; the EAGLE-3 speedup tracks this shift without degradation. Validation accuracy evolves indistinguishably under both decoding modes (Figure 2b), confirming that speculative decoding delivers a faster rollout engine without altering optimization behavior.



(a) Generation latency per training step. EAGLE-3 achieves mean speedups of $1.54\times$ (max $1.81\times$) on RL-Think and $1.79\times$ (max $2.85\times$) on RL-Zero. The RL-Zero baseline latency rises sharply in the first ~ 100 steps as the policy transitions from short outputs to longer reasoning traces; the EAGLE-3 gap persists throughout.



(b) Validation accuracy (AIME-2024) versus training step. The EAGLE-3 and autoregressive curves overlap closely throughout: RL-Think rises from ~ 0.60 to ~ 0.70 and RL-Zero from ~ 0.03 to ~ 0.33 , confirming that speculative decoding preserves the optimization trajectory.

Figure 2 | Training dynamics under autoregressive and speculative decoding for RL-Think (left) and RL-Zero (right). (a) Generation latency per step; (b) validation accuracy on AIME-2024. Speculative decoding reduces rollout latency throughout training while producing indistinguishable accuracy curves.

3.3. Analysis

The main results show that speculative decoding speeds up rollout generation without changing the learning trajectory. How much speedup is realized, however, depends on three practical decisions: how the draft model is initialized, how many tokens it proposes, and whether it is updated during training. We study each in turn, then examine the interaction with asynchronous execution as a separate axis.

Draft initialization. Table 3 shows that draft initialization strongly affects realized speedup. We train an EAGLE-3 draft on UltraChat and Magpie (Ding et al., 2023; Xu et al., 2025) to obtain a general chat-domain draft, and compare it against a draft trained on the same prompts with DAPO post-training data (Yu et al., 2025). At matched draft length $k = 3$, the DAPO initialization consistently outperforms the chat draft: speedup rises from $1.51\times$ to $1.77\times$ on RL-Zero and from $1.19\times$ to $1.53\times$ on RL-Think. Initialization quality is not only about generic drafting ability, but about alignment with the rollout distribution encountered during RL.

Initialization	RL-Zero		RL-Think	
	Acceptance	Speedup	Acceptance	Speedup
UltraChat	2.88	1.51×	2.40	1.19×
DAPO	3.32	1.77×	2.77	1.53×

Table 3 | Effect of draft initialization at fixed draft length $k = 3$ under offline drafting. In-domain initialization improves both acceptance length and realized speedup.

Draft length. Table 4 studies draft lengths $k \in \{3, 5, 7\}$. The smallest tested draft, $k = 3$, consistently gives the best end-to-end speedup. On RL-Zero, increasing the draft length raises acceptance from 3.32 to 4.35 and 5.06, but speedup falls from 1.77× to 1.44× and 1.21×. On RL-Think, speedup drops from 1.53× at $k = 3$ to 0.84× at $k = 5$ and 0.71× at $k = 7$, making the longer drafts slower than autoregressive decoding. Larger drafts increase speculative work enough to erase the benefit of higher acceptance, especially in harder regimes. This finding aligns with the speedup analysis in Section 2.2: the draft-length sweet spot is narrower than the acceptance-length curve would suggest.

Draft length k	RL-Zero		RL-Think	
	Acceptance	Speedup	Acceptance	Speedup
3	3.32	1.77×	2.77	1.53×
5	4.35	1.44×	3.23	0.84×
7	5.06	1.21×	3.48	0.71×

Table 4 | Effect of draft length. Acceptance length increases with k , but realized speedup declines. For RL-Think, $k \geq 5$ is slower than autoregressive decoding.

Online draft adaptation. We study online draft adaptation by reusing rollouts generated by the current policy as supervision for the EAGLE-3 draft. Concretely, when online adaptation is enabled, the draft reuses hidden-state and verifier-log-probability caches from the same MegatronLM forward pass that computes the GRPO loss, avoiding an additional policy recomputation. Table 5 shows that online updating provides limited additional gains when the draft is already well initialized: DAPO-initialized EAGLE-3 performs nearly identically offline (1.77×) and online (1.78×) on RL-Zero. The larger benefit appears for weaker initialization, where online adaptation improves speedup from 1.51× to 1.63× on RL-Zero and from 1.19× to 1.26× on RL-Think. Online draft maintenance thus serves as insurance against distribution mismatch rather than a general improvement strategy.

Variant	RL-Zero		RL-Think	
	Acceptance	Speedup	Acceptance	Speedup
UltraChat, offline	2.88	1.51×	2.40	1.19×
UltraChat, online	3.04	1.63×	2.55	1.26×
DAPO, offline	3.32	1.77×	2.77	1.53×
DAPO, online	3.29	1.78×	2.74	1.52×

Table 5 | Effect of online draft adaptation at fixed $k = 3$. Online updates help the weaker UltraChat initialization, but provide little additional benefit when the draft is already well aligned.

Interaction with asynchronous execution. The main experiments use synchronous RL to isolate rollout-engine effects. We next revisit the question under asynchronous execution by evaluating

RL-Think at policy lag 1 in a 16-node non-located configuration, with 12 nodes dedicated to generation and 4 nodes to training. In this regime, much of rollout generation is already hidden behind log-probability recomputation and policy updates, so the relevant quantity is not the full rollout latency but the training-side idle time caused by generation, namely the portion that remains exposed on the critical path. Speculative decoding reduces this exposed generation time from 10.4s to 0.6s per step and lowers effective step time from 75.0s to 60.5s ($1.24\times$). The learning trajectories remain similar under both decoding modes, with speculative decoding preserving the optimization behavior. The gain is smaller than in synchronous RL because asynchronous overlap already hides much of the rollout cost, but the result confirms that speculative decoding and asynchronous execution remain complementary.

4. Deployment Scale Projections

The experiments in Section 3 demonstrate $1.5\text{--}1.8\times$ generation speedup and up to $1.4\times$ overall step speedup at 8B scale under synchronous RL on 32 GPUs. A natural question is how these gains extrapolate: as model size, GPU count, and policy lag change, does speculative decoding remain effective? Furthermore, while our experimental results capture the overheads of the current software implementation, practitioners may want a software-agnostic view of the maximum speedup opportunity. To bridge this gap, we employ a high-fidelity performance simulator to project the benefits of speculative decoding across a broader deployment design space. The speedups reported in this section should be interpreted as opportunity envelopes, with emphasis on trends rather than absolute values.

4.1. Simulation methodology

A faithful simulation framework for RL rollouts must satisfy three requirements: (i) accurately represent device-level and system-level performance characteristics of the serving infrastructure, (ii) support a broad spectrum of model sharding strategies, and (iii) capture the long-tailed response length distributions characteristic of RL rollout workloads.

We utilize a proprietary GPU performance simulator that incorporates detailed models of GPU compute units, memory hierarchies, and interconnects. It leverages a kernel-aware analytical framework to evaluate state-of-the-art model partitioning strategies, including operator overlap and power-aware optimizations. A dynamic traffic generator estimates rollout batch sizes at each step based on a given response length distribution, enabling assessment of speculative decoding’s impact on long-tailed rollout latencies across both synchronous and asynchronous configurations.

We study models from the Qwen3 family using a rollout batch size of 4096, scaling to deployments of up to 2048 GB200 GPUs at FP8 precision.

4.2. Draft length and acceptance length sensitivity

One key lever for optimizing speedup is the draft length. The optimal choice depends on the achievable acceptance length, which is in turn determined by the dataset and task. Figure 3 presents a heatmap of expected speedups across a range of draft and acceptance lengths for Qwen3-235B-A22B in a synchronous RL deployment on 512 GPUs.

Longer draft lengths are beneficial only when workloads achieve high acceptance. For example, if 5 tokens are accepted on average at $k = 7$, the longer draft yields a $4.07\times$ rollout speedup but only a $1.96\times$ end-to-end speedup, since non-generation stages dilute the benefit. If acceptance remains at 3 regardless of draft length, $k = 3$ achieves a $2.72\times$ rollout speedup and a $1.70\times$ end-to-end

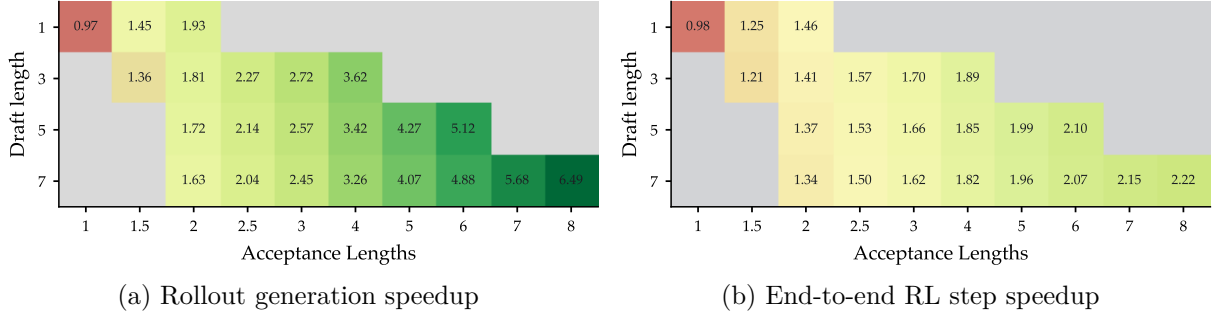


Figure 3 | Simulated rollout (a) and end-to-end (b) speedup for synchronous RL with Qwen3-235B-A22B on 512 GB200 GPUs, as a function of draft length (y -axis) and acceptance length (x -axis). Gray cells indicate infeasible configurations where acceptance exceeds $k + 1$. Non-generation stages bound the peak rollout gain of $6.49\times$ to $2.22\times$ end-to-end.

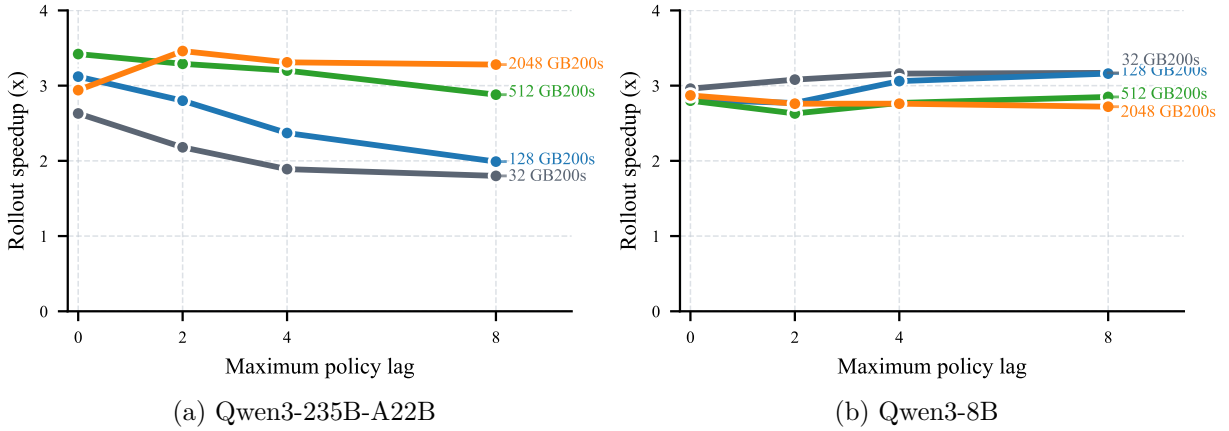


Figure 4 | Simulated rollout speedup across GPU count (32–2048 GB200s) and maximum policy lag (0–8) with draft length 5 and acceptance length 4. (a) Qwen3-235B-A22B exhibits strong sensitivity to deployment scale: speedup degrades with lag at smaller GPU counts (32, 128) but remains stable at 512–2048 GPUs. (b) Qwen3-8B is largely insensitive to both scale and lag, with all configurations achieving ~ 2.8 – $3.2\times$ rollout speedup.

speedup, a comparable end-to-end operating point with far lower speculative overhead. This mirrors the experimental finding in Table 4: higher acceptance length does not automatically justify longer drafts, and the gap between rollout and end-to-end gains grows with draft length.

4.3. Sensitivity to deployment scale and policy lag

Asynchronous RL reduces the effective rollout share on the critical path, which limits the opportunity for rollout-side acceleration. However, Figure 4 shows that substantial gains from speculative decoding remain achievable under realistic async settings. The effect of policy lag is scale-dependent: for the 235B model, smaller deployments (32, 128 GPUs) show pronounced speedup degradation as lag increases, while larger deployments (512, 2048 GPUs) maintain relatively stable rollout speedup even at lag 8. For the 8B model, all configurations are largely insensitive to lag. In all cases, speculative decoding provides meaningful improvement at typical operating lag values.

For sufficiently large models, larger deployments generally derive more benefit from speculation (Figure 4a). With a fixed rollout batch size, more GPUs result in a smaller local batch per model instance, exacerbating long-tail under-utilization and creating more room for speculative acceleration.

However, this relationship is not monotonic: at zero lag, the 2048-GPU deployment ($\sim 3.0\times$) underperforms 512 GPUs ($\sim 3.4\times$), likely because stretching the batch too thin forces suboptimal sharding. With modest async overlap (lag 2), the 2048-GPU configuration recovers and reaches the highest rollout speedup ($\sim 3.5\times$), as the additional pipeline concurrency compensates for the sharding inefficiency.

The 235B model (Figure 4a) shows markedly higher sensitivity to deployment scale and policy lag than the 8B model (Figure 4b), where all configurations cluster within a narrow $2.8\text{--}3.2\times$ band. Larger models occupy more GPUs per instance (e.g., 64 GPUs for 235B versus 8 for 8B in a 2048-GPU deployment) without compromising utilization, while smaller models partition the batch across more instances, fully exposing long-tail effects even at smaller scales. Practitioners training frontier-scale models should therefore pay attention to deployment configuration to maximize the benefit of speculative decoding. At the most favorable simulated operating point (Qwen3-235B-A22B, 2048 GPUs, lag 2), rollout speedup reaches $\sim 3.5\times$; combined with the high generation share characteristic of frontier-scale models, this translates to a projected $\sim 2.5\times$ end-to-end training speedup.

5. Related Work

RL post-training systems. Open RL post-training stacks now include NeMo-Aligner, OpenRLHF, veRL, and slime, which provide scalable orchestration and rollout-serving integration (Shen et al., 2024; Hu et al., 2025; Sheng et al., 2025; veRL Team, 2025; Zhu et al., 2025). Our work is narrower: rather than a new framework, we study speculative decoding as a deployable rollout primitive inside NeMo RL.

Rollout efficiency. Recent systems improve throughput through asynchronous generation, pipelined training, disaggregated engines, replay, importance-sampling corrections, and low-precision rollouts (Noukhovitch et al., 2025; Piché et al., 2025; Meta GenAI, 2025; Li et al., 2025a; Zheng et al., 2025a; Wang et al., 2025a; Sheng et al., 2026; Xi et al., 2026; Qiu et al., 2026; Li et al., 2026; Zheng et al., 2025b). Frontier model reports show that RL post-training typically combines several of these levers (DeepSeek-AI, 2024; Qwen Team, 2025; MiniMax Team, 2025a,b; Z.ai Team, 2025; DeepSeek-AI, 2025b; Kimi Team, 2026; Z.ai Team, 2026; NVIDIA, 2026). Speculative decoding is complementary: it accelerates rollout generation while preserving the target model’s sampling distribution.

Speculative decoding for RL. Speculative decoding was introduced as a lossless inference accelerator (Leviathan et al., 2023; Chen et al., 2023) and extended through tree verification, Medusa-style heads, and EAGLE drafting (Zhang et al., 2024; Cai et al., 2024; Miao et al., 2024; Li et al., 2024, 2025b; Gloeckle et al., 2024). Two recent papers apply it to RL specifically. FastGRPO (Zhang et al., 2025) focuses on concurrency-aware scheduling and online draft learning under high-concurrency group sampling. ReSpec (Chen et al., 2025) studies adaptive draft configurations and reward-weighted drafter adaptation. Our paper differs in focus. We study end-to-end systems integration, including verifier-exact rollout acceleration inside a production-grade RL stack, coordinated weight synchronization, and analysis of how speculation composes with synchronous and asynchronous execution across scales.

6. Conclusion

We integrated speculative decoding into NeMo RL as a rollout acceleration primitive that preserves verifier-exact training semantics. On 8B-scale reasoning workloads under synchronous RL, EAGLE-3

speculative decoding reduces rollout generation latency by $1.5\text{--}1.8\times$ and overall RL step time by up to $1.41\times$, with no change in validation accuracy. Simulator projections show that these gains grow at deployment scale: at favorable operating points for a 235B model, rollout speedups exceed $3\times$ and projected end-to-end training speedup reaches approximately $2.5\times$. Draft initialization quality, draft length, and generation share are the primary determinants of realized gain, and speculative decoding composes with asynchronous execution as a complementary mechanism.

References

- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5209–5235. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. ReSpec: Towards optimizing speculative decoding in reinforcement learning systems. *arXiv preprint arXiv:2510.26475*, 2025. URL <https://arxiv.org/abs/2510.26475>.
- DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL <https://arxiv.org/abs/2412.19437>.
- DeepSeek-AI. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025a. doi: 10.1038/s41586-025-09422-z. URL <https://www.nature.com/articles/s41586-025-09422-z>.
- DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025b. URL <https://arxiv.org/abs/2512.02556>.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, 2023. doi: 10.18653/v1/2023.emnlp-main.183. URL <https://aclanthology.org/2023.emnlp-main.183/>.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. URL <https://proceedings.mlr.press/v235/gloeckle24a.html>.
- Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. OpenRLHF: A Ray-based easy-to-use, scalable and high-performance RLHF framework. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2025. URL <https://aclanthology.org/2025.emnlp-demos.48/>.

- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O. Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. In *Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Rwhi91ideu>.
- Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026. URL <https://arxiv.org/abs/2602.02276>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Siheng Li, Zhanhui Zhou, Wai Lam, Chao Yang, and Chaochao Lu. RePO: Replay-enhanced policy optimization. *arXiv preprint arXiv:2506.09340*, 2025a. URL <https://arxiv.org/abs/2506.09340>.
- Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, and Brucek Khailany. QuRL: Efficient reinforcement learning with quantized rollout. *arXiv preprint arXiv:2602.13953*, 2026. URL <https://arxiv.org/abs/2602.13953>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28935–28948. PMLR, 2024. URL <https://proceedings.mlr.press/v235/li24ag.html>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025b. URL <https://arxiv.org/abs/2503.01840>.
- Meta GenAI. LlamaRL: A distributed asynchronous reinforcement learning framework for efficient large-scale LLM training. *arXiv preprint arXiv:2505.24034*, 2025. URL <https://arxiv.org/abs/2505.24034>.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, et al. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’24)*, pages 932–949, 2024. doi: 10.1145/3620666.3651335. URL <https://dl.acm.org/doi/10.1145/3620666.3651335>.
- MiniMax Team. MiniMax-M1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025a. URL <https://arxiv.org/abs/2506.13585>.
- MiniMax Team. MiniMax-M2 & Agent: Ingenious in simplicity. <https://minimax.io/news/minimax-m2>, 2025b.
- Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous RLHF: Faster and more efficient off-policy RL for language models. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=FhTAG591Ve>.
- NVIDIA. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. *arXiv preprint arXiv:2604.12374*, 2026. URL <https://arxiv.org/abs/2604.12374>.

- Alexandre Piché, Ehsan Kamalloo, Rafael Pardini, Xiaoyin Chen, and Dzmitry Bahdanau. Pipelin-eRL: Faster on-policy reinforcement learning for long sequence generation. *arXiv preprint arXiv:2509.19128*, 2025. URL <https://arxiv.org/abs/2509.19128>.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Xinyue Yang, Jiadai Sun, Yu Yang, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=oVKEAFjEqv>.
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=eOLdGbXT6t>.
- Zhaopeng Qiu, Shuang Yu, Jingqi Zhang, Shuai Zhang, Xue Huang, Jingyi Yang, and Junjie Lai. FP8-RL: A practical and stable low-precision stack for LLM reinforcement learning. *arXiv preprint arXiv:2601.18150*, 2026. URL <https://arxiv.org/abs/2601.18150>.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Gerald Shen, Zhilin Wang, Olivier Delalleau, Jiaqi Zeng, Yi Dong, Daniel Egert, Shengyang Sun, Jimmy Zhang, Sahil Jain, Ali Taghibakhshi, et al. NeMo-Aligner: Scalable toolkit for efficient model alignment. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=yK2eGE8QVW>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3696075. URL <https://dl.acm.org/doi/10.1145/3689031.3696075>.
- Yuepeng Sheng, Yuwei Huang, Shuman Liu, Anxiang Zeng, and Haibo Zhang. ESPO: Entropy importance sampling policy optimization. *arXiv preprint arXiv:2512.00499*, 2026. URL <https://arxiv.org/abs/2512.00499>.
- veRL Team. veRL: Volcano engine reinforcement learning for LLMs. <https://github.com/volcengine/verl>, 2025.
- Jiakang Wang, Runze Liu, Lei Lin, Wenping Hu, Xiu Li, Fuzheng Zhang, Guorui Zhou, and Kun Gai. ASPO: Asymmetric importance sampling policy optimization. *arXiv preprint arXiv:2510.06062*, 2025a. URL <https://arxiv.org/abs/2510.06062>.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Monica Lam, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025b. URL <https://arxiv.org/abs/2504.20073>.

- Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, and Ligeng Zhu. Jet-RL: Enabling on-policy FP8 reinforcement learning with unified training and rollout precision flow. *arXiv preprint arXiv:2601.14243*, 2026. URL <https://arxiv.org/abs/2601.14243>.
- Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. Magpie: Alignment data synthesis from scratch by prompting aligned LLMs with nothing. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=Pnk7vMbznK>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Z.ai Team. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. *arXiv preprint arXiv:2508.06471*, 2025. URL <https://arxiv.org/abs/2508.06471>.
- Z.ai Team. GLM-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11263–11282, 2024. doi: 10.18653/v1/2024.acl-long.607. URL <https://aclanthology.org/2024.acl-long.607/>.
- Yizhou Zhang, Ning Lv, Teng Wang, and Jisheng Dang. FastGRPO: Accelerating policy optimization via concurrency-aware speculative decoding and online draft learning. *arXiv preprint arXiv:2509.21792*, 2025. URL <https://arxiv.org/abs/2509.21792>.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025a. URL <https://arxiv.org/abs/2507.18071>.
- Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts. In *Advances in Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=x5lITYXmW2>.
- Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An LLM post-training framework for RL scaling. <https://github.com/THUDM/slime>, 2025.